

Production Verification RUNBOOK — Privacy Incident (July 2026)

Runnable companion to docs/PROD_VERIFICATION_CHECKLIST_2026-07.md . Everything below is copy-paste. The **only** things the operator supplies are in the "FILL IN" block. All table/column/route names have been verified against the committed schema (see "Schema verification" at the bottom).

*Run order: **1a** → **1b** (seed first if needed) → **1c** → **2** → **3** → **4**. Do the seed BEFORE 1b/1c if prod has no naturally-barred candidate to test against, and run the CLEANUP at the very end.*

FILL IN (the only operator-supplied values)

```
# ---- Host ----
export PROD_HOST="app.example.com"                # no scheme, no trailing slash

# ---- Tokens (Bearer, without the word 'Bearer') ----
export TOK_NOPOOL="..."      # employer-tenant user WITHOUT candidate-database access
export TOK_POOL="..."        # employer-tenant user WITH candidate-database access

# ---- Target ids ----
export TENANT_ID="..."       # the WITH-access tenant's id (used in the browse URL)
export BARRED_CANDIDATE_ID="..." # a platform-pool candidate that IS barred (see 1b seed)
export CONTROL_CANDIDATE_ID="..." # a platform-pool candidate that is COMPLIANT (should appear)
```

DB SQL below is run in a **prod** psql/console session (Items 1c, 2, 3, and the seed). The curl blocks are run from any shell with the vars above exported.

Item 1a — endpoints live + licence-less tenant sees no platform pool

```
# HTTP status only (expect 200)
curl -sS -o /dev/null -w 'status=%{http_code}\n' \
  -H "Authorization: Bearer ${TOK_NOPOOL}" \
  "https://${PROD_HOST}/api/candidates"

# Count any platform-pool rows leaking to a licence-less tenant (expect 0)
curl -sS -H "Authorization: Bearer ${TOK_NOPOOL}" \
  "https://${PROD_HOST}/api/candidates" \
  | grep -o '"pool": "platform"' | wc -l
```

- **SAFE:** status 200 AND the platform-pool count is 0 .
- **ESCALATE:** any "pool": "platform" for the no-pool tenant.

Item 1b — the seal bars a *barred* candidate (and still returns a control)

If prod has no barred platform-pool candidate to point at, run **SEED FOR 1b** below first, set `BARRED_CANDIDATE_ID` / `CONTROL_CANDIDATE_ID` to the seeded ids, then run these. **Run CLEANUP at the end.**

```
# (a) barred candidate must NOT be in the shared browse -> expect 0
curl -sS -H "Authorization: Bearer ${TOK_POOL}" \
  "https://${PROD_HOST}/api/tenants/${TENANT_ID}/candidate-database" \
  | grep -c "${BARRED_CANDIDATE_ID}"

# (b) compliant control MUST be in the shared browse -> expect >= 1
curl -sS -H "Authorization: Bearer ${TOK_POOL}" \
  "https://${PROD_HOST}/api/tenants/${TENANT_ID}/candidate-database" \
  | grep -c "${CONTROL_CANDIDATE_ID}"

# (c) by-id detail must 404 for the barred candidate -> expect 404 twice
curl -sS -o /dev/null -w 'career-profile=%{http_code}\n' \
  -H "Authorization: Bearer ${TOK_POOL}" \
  "https://${PROD_HOST}/api/candidates/${BARRED_CANDIDATE_ID}/career-profile"
curl -sS -o /dev/null -w 'career-recording=%{http_code}\n' \
  -H "Authorization: Bearer ${TOK_POOL}" \
  "https://${PROD_HOST}/api/candidates/${BARRED_CANDIDATE_ID}/career-recording"
```

- **SAFE:** (a) 0 , (b) >= 1 , (c) both 404 .
- **ESCALATE:** barred appears in (a), or (c) returns 200 .

SEED FOR 1b — throwaway barred + compliant twin (run in prod DB)

`candidates.id` has **no DB default** (app mints it), so we supply it. `tenant_id` is NOT NULL ; any existing tenant id works — `pool = 'platform'` is what makes the row cross-tenant-visible in the shared browse. `discovery_paused = true` is the cleanest **unconditional** bar (hides from ALL discovery, no domain/job dependency).

```
-- pick any existing tenant to "home" the throwaway rows:
-- SELECT id FROM tenants LIMIT 1; -- copy one id into :HOME_TENANT below

-- BARRED canary (discovery_paused = true)
INSERT INTO candidates
(id, tenant_id, first_name, last_name, email, pool, discovery_paused)
VALUES
('privacy-canary-barred-DELETE-ME', :HOME_TENANT,
 'ZZ-Canary', 'Barred', 'privacy-canary-barred@delete.me', 'platform', true);

-- COMPLIANT control twin (no bars) — proves the surface isn't just empty
INSERT INTO candidates
(id, tenant_id, first_name, last_name, email, pool)
VALUES
```

```
('privacy-canary-control-DELETE-ME', : 'HOME_TENANT',  
  'ZZ-Canary', 'Control', 'privacy-canary-control@delete.me', 'platform');
```

Then:

```
export BARRED_CANDIDATE_ID="privacy-canary-barred-DELETE-ME"  
export CONTROL_CANDIDATE_ID="privacy-canary-control-DELETE-ME"
```

Alternative harder bars (swap the flag in the barred INSERT if you want to test a compliance bar instead of a discovery bar): `do_not_contact = true` , or `data_erased_at = now()` . Relational bars (`match_only_visibility` , `blocked_company_domains`) depend on the querying tenant's jobs/domain, so `discovery_paused` is preferred for a deterministic one-row test.

CLEANUP (run after Items 1-4 are done)

```
DELETE FROM candidates  
WHERE id IN ('privacy-canary-barred-DELETE-ME', 'privacy-canary-control-DELETE-ME');
```

Item 1c — rec-push never pushed a barred candidate (run in prod DB)

All column names verified against `talent_pool_submissions` and `candidates` .

```
SELECT s.id, s.candidate_id, s.client_tenant_id, s.pushed_at  
FROM talent_pool_submissions s  
JOIN candidates c ON c.id = s.candidate_id  
WHERE c.do_not_contact = true  
      OR c.data_erased_at IS NOT NULL  
      OR c.discovery_paused = true  
      OR c.hide_from_current_employer = true  
      OR c.match_only_visibility = true  
      OR jsonb_array_length(c.blocked_company_domains) > 0  
ORDER BY s.pushed_at DESC  
LIMIT 100;
```

- **SAFE:** zero rows.
- **ESCALATE:** any row — capture `client_tenant_id` + `pushed_at` for scope.

*Note: `match_only_visibility` and `blocked_company_domains` are relational bars (a match-only candidate may legitimately be pushed to a tenant whose role matches). Treat hits on those two as **review**, and `do_not_contact` / `data_erased_at` / `discovery_paused` / `hide_from_current_employer` as **hard escalate**.*

Item 2 — access-log review (run in prod DB)

Verified: `http_access_logs` stores `method` and `route_pattern` as **separate** columns;
`route_pattern` is the **registered pattern only** (`:tenantId` , `:candidateId` placeholders), never the raw URL.

```
-- First confirm the exact stored patterns (sanity check):
SELECT DISTINCT route_pattern
FROM http_access_logs
WHERE route_pattern LIKE '%candidate%';

-- Coverage start (nothing before this is queryable):
SELECT min(occurred_at) AS logging_since FROM http_access_logs;

-- The review query:
SELECT occurred_at, method, route_pattern, status_code, user_id, tenant_id
FROM http_access_logs
WHERE method = 'GET'
      AND route_pattern IN (
        '/api/candidates',
        '/api/tenants/:tenantId/candidate-database',
        '/api/candidates/:candidateId/career-recording',
        '/api/candidates/:candidateId/career-profile'
      )
ORDER BY occurred_at DESC
LIMIT 500;
```

- **SAFE:** hits only from tenants that legitimately have pool access; by-id career routes show 404 for barred candidates.
- **ESCALATE:** by-id career routes returning 200 to a tenant that shouldn't see that candidate; or list/browse hits from licence-less tenants.
- **HONEST LIMIT:** the log proves *which route / which tenant*, **not which candidate row** was returned (no id/body/query stored). Correlate with Item 1c output, not the log alone. A clean log = "no *logged* hits since logging_since ," not "never happened."

Item 3 — actual RLS in prod (run in prod DB as table owner/superuser)

Verified catalog names. Run per Class-A table (candidates , applications , interview_sessions , sourced_candidates , ...). Replace the literal below.

```
-- 3a. Is RLS enabled AND forced?
SELECT relname, relrowsecurity, relforcerowsecurity
FROM pg_class
WHERE relname = 'candidates';

-- 3b. Does the tenant-isolation policy actually exist? (raw catalog form)
SELECT polname, polcmd,
       pg_get_expr(polqual, polrelid) AS using_expr,
       pg_get_expr(polwithcheck, polrelid) AS check_expr
FROM pg_policy
WHERE polrelid = 'candidates'::regclass;
```

Easier-to-read equivalent for 3b if the raw catalog is awkward (same data via the built-in view):

```
SELECT polycyname, cmd, qual, with_check
FROM pg_policies
WHERE tablename = 'candidates';
```

- **SAFE:** 3a = (t, t) AND 3b returns ≥1 policy (expected tenant_isolation) with expression current_setting('app.is_platform_admin', true) = 'true' OR tenant_id = current_setting('app.current_tenant_id', true) .
- **ESCALATE (drizzle-push failure mode):** 3b returns **zero policies** on a Class-A table → isolation NOT DB-enforced. (t, 0 policies) = deny-all; (f, ...) = unfiltered/app-code-only. Remediate with the .sql policy migration, **not** drizzle-kit push .
- **Class B tables** (candidate_job_intelligence , hiring_manager_shares , interview_summaries , candidate_career_profiles , ...) are **expected** to show (f, f) + zero policies — by design; the app-layer seal + check:classb-read CI guard are the whole control. Do not escalate Class-B (f,f) .

Item 4 — sealed code actually PUBLISHED to prod

4a. In the deploy console, read the currently-serving build's source commit and confirm it is **at or after 2026-07-07 00:42 UTC** ("Privacy incident Phase 3").

- SAFE: deployed commit ≥ that commit. ESCALATE: older → redeploy before closing.

4b. Behavioural: Item 1b returning 404 on the by-id career routes in prod is direct evidence the sealed path is live. 4a (SHA) and 1b (behaviour) must agree.

There is no build-SHA HTTP endpoint to curl (/health , /healthz , /healthz/live report liveness only). 4a depends on the deploy console until a /version endpoint is added.

Schema verification (checked against committed schema this pass)

Thing the checklist asserts	Verified?	Source
route_pattern = '/api/candidates'	☐	candidatesRouter on main router mounted at /api; router.get("/candidates")
route_pattern = '/api/tenants/:tenantId/candidate-database'	☐ (param is :tenantId, not :id)	tenants.ts:699 router.get("/tenants/:tenantId/candidate-database")
route_pattern = '/api/candidates/:candidateId/career-recording'	☐	candidates.ts:2644
route_pattern = '/api/candidates/:candidateId/career-profile'	☐	candidates.ts:2736
http_access_logs has separate method + route_pattern (registered pattern, no raw URL/id)	☐	lib/db/src/schema/http-access-logs.ts; buildAccessLogRow

candidate bar columns do_not_contact, data_erased_at, discovery_paused, hide_from_current_employer, match_only_visibility, blocked_company_domains	□	lib/db/src/schema/candidates.ts:66–107
talent_pool_submissions.candidate_id / client_tenant_id / pushed_at	□	lib/db/src/schema/talent_pool.ts
pg_class(relrowsecurity, relforcerowsecurity) + pg_policy(polqual, polwithcheck) via pg_get_expr	□ (standard catalogs; pg_policies view offered as fallback)	PostgreSQL system catalogs
candidates.id has no DB default (seed must supply id); tenant_id NOT NULL	□	candidates.ts:31–33 (\$defaultFn is app- side only)

One correction to the original checklist prose: Item 1's endpoint list writes `GET /api/tenants/:id/candidate-database`, but the registered param is `:tenantId`. Harmless for curl (you pass a real id), but the **log query in Item 2 must use `:tenantId`** (it does). No other discrepancies found.